

# REPAIRLENS: PRODUCTIVE DIAGNOSIS FOR PARTIALLY OBSERVED REPAIR DECISIONS

*Author information required before submission*

Controlled-model research draft, 12 September 2026

## ABSTRACT

After a known source error is corrected, incomplete provenance leaves uncertainty about which derived artifacts need repair. Full diagnosis may distinguish hypotheses that imply the same repair, while a diagnostic replay may already compute an artifact needed for recovery. We formulate this interaction as productive diagnosis over a finite behavior family, with posterior support, reusable artifacts and used probes as the state. A Bellman reference and a two-step controller trade information acquisition against remaining repair cost. On 120 deterministic synthetic signal workflows, both restore all declared state checks; the two-step controller reduces assigned operation cost by 23.4% relative to restart. This is not an end-to-end speedup: measured repair-phase time is 1.84 ms versus 0.59 ms for restart. Removing the true repair mask from the candidate family lowers restoration to 38.3%. The study establishes a controlled cost mechanism, not general LLM-agent recovery.

**Index Terms**— active diagnosis, sequential decision making, replay, provenance, signal workflows

## 1. INTRODUCTION

After a source correction, incomplete dependency records obscure which derived artifacts remain valid. Recomputing all descendants is conservative but wasteful; trusting one likely dependency model can preserve invalid state. We study gathering evidence before selecting a repair.

This is not a new general argument for decision-theoretic diagnosis. Classical troubleshooting already couples observations, uncertainty and repair cost [1, 2]. Bayesian active learning and equivalence-class determination distinguish hypothesis identification from decision-relevant testing [3]. In agent systems, REVISE conservatively expands recovery when provenance is incomplete [4]; Causal Agent Replay estimates intervention effects [5]; and minimal repair family recovery searches for inclusion-minimal event sets within a declared graph domain [6]. Repair or Resample further demonstrates why the pre-intervention history must be held fixed when attributing recovery to a repair [7].

We study a bounded bridge between these ideas. A replay can have two outputs: an observation about dependency

structure and a newly valid artifact. If the second output is discarded by the cost model, the preferred diagnostic action can be wrong even when its information estimate is right. Conversely, identifying provenance details that do not change the repair can consume observations without improving the recovered state. We call the resulting computation *productive diagnosis*: diagnosis and repair share validated products, and stopping depends on the cost of a support-valid repair rather than on identifying one complete graph.

Our contributions are a finite-state cost formulation with explicit cache state; a two-step controller evaluated against its exact Bellman reference and information-driven baselines; and an executable recovery audit that reports both assigned operation cost and actual local runtime. The artifact intentionally exposes two boundaries: model support omissions can invalidate restoration, and planning overhead can erase operation savings. These results are mechanism evidence, not a claim that arbitrary language-agent failures have been solved.

## 2. MODEL AND RECOVERY OBJECTIVE

### 2.1. Candidate behavior, not merely a graph

Let  $\mathcal{H}$  be a finite family of dependency-behavior hypotheses with positive prior  $p(h)$ . Each hypothesis supplies both a set  $D(h)$  of artifacts requiring recomputation after a known source correction and the outcome  $o_u(h)$  of each admissible diagnostic probe  $u$ . A graph alone is insufficient to predict an intervention. The actual  $h^*$  is held by the execution driver, never passed to the policy. Initial support contains  $h^*$  in the primary experiment; we explicitly violate this assumption in a stress test.

An artifact  $j$  has recomputation cost  $c_j > 0$ . A probe has cost  $d_u > 0$  and may materialize a set  $A_u$  of artifacts under the corrected inputs. These products are reusable only if their input version, content integrity and scope match the final repair. The finite model assumes they do. The current policy state is  $(B, C, U)$ : retained hypotheses, valid cached artifacts and already used probes. Deterministic observations restrict the posterior to  $B_o^u = \{h \in B : o_u(h) = o\}$ . Used probes are not repeated.

The permitted terminal action recomputes a superset of all potentially damaged artifacts, excluding products already

available. Its cost is

$$R(B, C) = \sum_{j \in (\bigcup_{h \in B} D(h)) \setminus C} c_j. \quad (1)$$

This is a conservative repair contract over the declared family. It neither reconstructs a unique graph nor searches arbitrary program patches. If  $B$  is empty, it cannot certify local reuse; the implementation falls back to a complete restart.

## 2.2. Cost and correctness are separate

The objective is expected *diagnosis plus remaining repair* cost, including common source correction and publication checks. A completed repair must satisfy the current result, all declared persistent-state checks and two downstream derived checks. Correcting only the last answer does not count. The evaluator is invoked after the policy stops; its corrected outputs and true dependency mask cannot select probes, update support or terminate the search.

Prices in this study are assigned normalized operation units. We measure planning and local execution separately. Therefore an improvement in this objective is not automatically a wall-time or monetary improvement. The distinction is consequential because the executable workers are deliberately cheap.

## 3. PRODUCTIVE DECISION-DIRECTED REPLAY

### 3.1. Bellman reference and two-step controller

Given a retained support, the conditional probability of observation  $o$  is

$$p(o \mid B, u) = \frac{\sum_{h \in B_o^u} p(h)}{\sum_{h \in B} p(h)}. \quad (2)$$

Write  $s = (B, C, U)$  and  $s_{u,o} = (B_o^u, C \cup A_u, U \cup \{u\})$ . The exact recursion is

$$Q_u(s) = d_u + \sum_o p(o \mid B, u) V(s_{u,o}), \quad (3)$$

$$V(s) = \min\{R(B, C), \min_{u \notin U} Q_u(s)\}. \quad (4)$$

A probe with no support refinement and no new reusable artifact is dominated. Positive costs and one-use probes make the recursion finite. Memoization provides an exact reference for the small experimental family. RepairLens-2 truncates the lookahead at two probes, uses  $R$  as the terminal value, executes the first selected probe and replans. A one-step version is the myopic baseline. Ties stop rather than buy an observation.

**Proposition 1.** Equation (4) gives minimum expected assigned cost among policies using the declared probes and terminal repair contract. If  $h^* \in B$  throughout, all reused products are valid and terminal recomputation is faithful, the final declared artifact state is correct.

*Proof.* Induct on the number of unused probes. With none left, (1) is the required support-valid terminal cost. Otherwise a policy either stops or selects one unused probe, pays its cost, observes one posterior branch and continues. Minimizing over these exhaustive first choices yields (4). For correctness, a deterministic observation cannot remove  $h^*$  from a correctly specified family. Thus  $D(h^*)$  is covered by the terminal union or valid cache, and every required artifact is available under the corrected inputs.  $\square$

This is conditional dynamic programming, not a new general theorem about causality or agent safety. Worst-case support and cache enumeration is exponential; the exact reference is not presented as a scalable deployed controller. Two-step lookahead has no claimed approximation factor.

### 3.2. The product changes the diagnostic choice

Consider two equally likely hypotheses requiring repair of artifact A or B, each at cost 10. A diagnostic-only test identifies the hypothesis for cost 6. A productive refresh of A gives the same observation for cost 10 and leaves A available. The diagnostic-first policy costs  $6 + 10 = 16$ . Refreshing A first costs  $10 + 0.5(10) = 15$ , because only the B case needs another repair. If refresh products are discarded, diagnostic-first is optimal instead. The implementation verifies both values exactly.

More generally, a posterior alone is not sufficient for this objective: identical supports with different valid caches can have different optimal next actions. Additional valid cached products cannot increase the optimal assigned cost, since an old feasible policy remains available and can omit redundant terminal computations. This monotonicity relies on free, valid cache retention; it does not cover cache eviction, inconsistent snapshots or unpriced validation.

### 3.3. Decision sufficiency versus full identification

Suppose  $2^n$  equally likely hypotheses differ only in  $n$  independent provenance-route bits, while every hypothesis requires the same damaged block set. If available route probes expose one bit at a time, exact graph identification requires all  $n$  bits in the worst case. A repair-directed policy requires zero diagnostic probes and immediately performs the common repair. This is a construction under a specified probe family, not a universal  $\log K$  complexity bound based only on the number of repair classes.

Route uncertainty cannot always be discarded. In the coupled family below, an inspection result is XORed with a route bit; observing that bit can be necessary to interpret a damage signal. We retain full candidate behavior while deciding when to stop, rather than merging equal-repair hypotheses without checking their observation models. This preserves useful nuisance information without forcing its complete recovery.

### 3.4. Executable artifacts and fidelity

Each workflow begins with the same deterministic signal data, source value and initial artifact bytes. Per-method replicas reconstruct this identical prefix; their digests must match. After changing the known source from zero to one, selected probes really execute the worker. Materialized results are written atomically as JSON envelopes containing an input-scope digest and content checksum. Reuse checks both. Snapshot restoration, source-version mismatch and content-tamper detection have separate unit tests.

These checks protect the declared local computation against accidental mismatch. A checksum is not source authentication or a defense against a malicious host. The experiments do not reproduce inaccessible LLM internals or undo external side effects. Preserved artifacts are rebound to the current source scope only when all retained hypotheses establish independence; missing true hypotheses can make that decision wrong.

## 4. EXPERIMENTS AND RESULTS

### 4.1. Controlled executable workload

We generate 120 independent cases, 20 in each of six pre-declared families. A case contains three synthetic 64-sample signals and three output blocks, each storing a mean, root-mean-square value and DC magnitude from a real NumPy Fourier transform. A hidden nonempty damage mask determines which workers depend on the corrected source. Two route bits encode alternative provenance paths. Candidate probabilities are drawn at the mask level from a Dirichlet distribution with concentration 0.7, then divided uniformly across routes; the true behavior is sampled from this public prior.

Sparse cases allow one damaged block; overlap cases allow two; general cases allow all nonempty masks. Nuisance cases share one damage mask across four routes. Coupled cases require route decoding for inspection. Costly-inspection cases price nonproductive inspection at 2.4 times the corresponding block refresh. There are at most 28 hypotheses and nine probes: three productive refreshes, three inspections, one two-block parity query and two route queries. Block prices are integers from 3 to 9; the code records every other price. These controlled operators are not natural agent failures or a public benchmark.

Eleven methods start from the same failed state: restart; conservative union repair; repair under the highest-prior hypothesis; random informative probing; full-hypothesis entropy; repair-class entropy; an EC2-style cross-class edge-cut score; one-step lookahead; exact planning without reuse; RepairLens-2; and exact productive planning. All share hypotheses, probes, candidate repairs and observable outcomes. The information baselines are disclosed local finite-model

**Table 1.** 120 cases per policy. Complete is a percentage; cost is normalized by each case’s restart cost. Planning time is measured, not assigned.

| Policy          | Complete | Cost  | Queries | Plan ms |
|-----------------|----------|-------|---------|---------|
| Restart         | 100.0    | 1.000 | 0.00    | 0.00    |
| Conservative    | 100.0    | 0.951 | 0.00    | 0.00    |
| Point estimate  | 67.5     | 0.603 | 0.00    | 0.00    |
| Random          | 100.0    | 1.065 | 2.67    | 0.07    |
| Graph entropy   | 100.0    | 0.851 | 3.87    | 0.26    |
| Class entropy   | 100.0    | 0.828 | 1.88    | 0.17    |
| EC2-style       | 100.0    | 0.850 | 3.54    | 0.27    |
| Myopic          | 100.0    | 0.838 | 0.53    | 0.13    |
| Discard replay  | 100.0    | 0.822 | 1.15    | 14.32   |
| RepairLens-2    | 100.0    | 0.766 | 1.77    | 1.13    |
| Exact reference | 100.0    | 0.755 | 1.92    | 26.35   |

instantiations, not reproduced CAR, REVISE or MRFR systems. The no-reuse ablation discards diagnostic products before final repair.

### 4.2. Operation savings and their mechanism

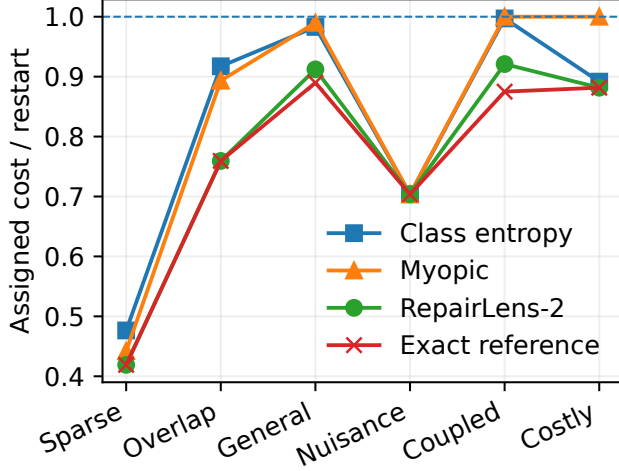
Table 1 summarizes 1,320 actual workflow runs. All support-valid methods restore every primary case; the point-estimate policy restores only 67.5%. Since complete support and faithful deterministic workers are constructed conditions, the 100% values primarily verify implementation of the contract, not deployment robustness.

Exact productive planning reduces normalized assigned cost by 24.5% [21.8,27.3] relative to restart. RepairLens-2 saves 23.4% [20.8,26.1]. Intervals use 3,000 paired bootstrap replicates within each fixed family, with equal family weights. They are descriptive conditional on the generator. Relative to class entropy, two-step planning reduces normalized cost by 0.0621 [0.0393, 0.0857]. Its cost exceeds the exact reference by 0.0114 [0.0042, 0.0188]. Exact reuse improves over exact discard by 0.0672 [0.0501, 0.0843], isolating the productive-artifact mechanism.

Each operation is charged once in a trace ledger. Materialized refresh counts exclude nonproductive inspection computations; those inspections still have assigned costs and contribute to measured execution time. Common source correction costs one unit and publication validation costs 0.15 per block. Hidden evaluation and common prefix construction are not optimization feedback and are outside the repair-phase timing.

### 4.3. Planning overhead reverses the latency conclusion

Mean measured repair-phase time, including planning, is 27.24 ms for exact planning and 1.84 ms for RepairLens-2, versus 0.59 ms for restart. The exact method evaluates about 3,617 memoized states per case on average; two-step plan-



**Fig. 1.** Assigned operation cost by workload family. The dashed line is restart. Nuisance and coupled cases distinguish irrelevant routes from routes needed to interpret a probe.

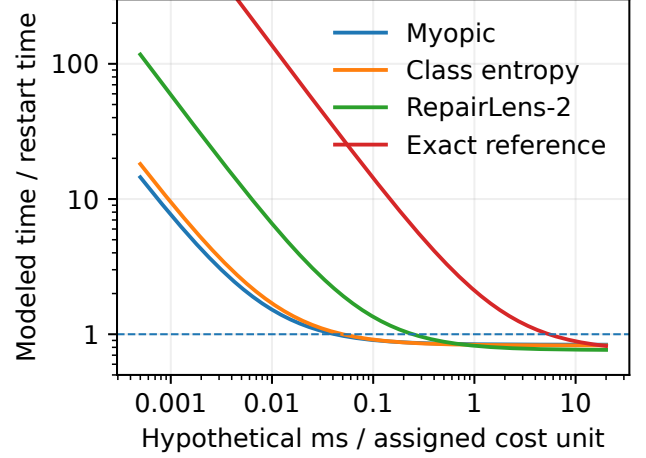
ning evaluates about 222. On these cheap local workers, neither is an end-to-end latency improvement. Reporting only operation counts would conceal this negative result.

To separate measured evidence from a cost assumption, Figure 2 considers the sensitivity model  $T_m(\tau) = t_m^{\text{plan}} + \tau \bar{c}_m$ , where  $\tau$  is a hypothetical latency per declared operation-cost unit. Under this model, the two-step break-even against restart is about 0.246 ms per unit and the exact break-even is 5.45 ms. These are algebraic thresholds using observed planning time and assigned costs, not measured API latencies, cloud savings or predictions validated on expensive tasks. The real application must supply its own cost model and measurements.

#### 4.4. Missing support is a substantive failure

An additional 60-case paired stress test removes every candidate with the true damaged-block mask. Exact planning restores 100% with covered support but only 38.3% after this removal. An empty-support fallback triggers in 15% of omission cases; many incorrect families remain observationally compatible with the probes actually selected. Thus detecting an impossible observation does not protect against all missing-model failures. This stress test deliberately has a 100% omission rate and is not an estimate of deployment prevalence.

Downstream checks aggregate means and weighted RMS values. They expose stale state but are derived from the same workflow, not independent unseen tasks. Their passing does not establish a distribution-free recovery guarantee.



**Fig. 2.** Hypothetical latency sensitivity, not a measured speedup. Planning is amortized only if real operation latency is sufficiently large.

## 5. LIMITATIONS AND CONCLUSION

RepairLens is a finite mechanism study. Damage masks, probe response models and priors are supplied, not discovered from unrestricted traces. Workers are deterministic, the source fault is known, and only three recomputation blocks are present. There are no LLM calls, natural failure cases, learned provenance extractors, concurrent effects or independently audited labels. Exact dynamic programming has exponential state growth. The controlled comparison does not establish superiority to full contemporary agent systems, and the observed local runtime is worse than restart.

Within this boundary, the results identify a useful design distinction: information alone does not determine the value of a diagnostic replay; validated reusable products and the remaining repair bill also matter. The next research gate is an expensive, independently specified workflow where these savings survive total runtime accounting and imperfect candidate coverage. The present artifact provides executable policies, paired traces, state checks, source hashes and negative controls for that test, without treating a controlled prototype as a submission-ready general recovery system.

## 6. ACKNOWLEDGMENTS AND AI DISCLOSURE

ChatGPT (OpenAI) assisted with the formulation, experimental code, plotting code and text in all sections. Measurements were produced by executing the supplied code. No independent human validation is claimed. Human authorship and review are required before submission.

## 7. REFERENCES

- [1] D. Heckerman, J. S. Breese, and K. Rommelse, “Troubleshooting under uncertainty,” Microsoft Research, Tech. Rep. MSR-TR-94-07, 1994.
- [2] J. S. Breese and D. Heckerman, “Decision-theoretic troubleshooting: A framework for repair and experiment,” arXiv:1302.3563, archival preprint, 2013.
- [3] D. Golovin, A. Krause, and D. Ray, “Near-optimal Bayesian active learning with noisy observations,” in *Advances in Neural Information Processing Systems*, vol. 23, 2010, pp. 766–774.
- [4] R. Qi, X. Wu, P. Liu, J. Chen, and Y. Liu, “REVISE: Validity-guided recovery for online revisions in agent workflows,” arXiv:2609.00643, 2026.
- [5] J. Shah, “Causal agent replay: Counterfactual attribution for LLM-agent failures,” arXiv:2606.08275, 2026.
- [6] B. Li, Y. Song, Z. Yao, and T. Li, “Localizing emergent failures in agentic AI: Recovering minimal repair families via counterfactual replay,” arXiv:2608.29228, 2026.
- [7] Z. Luan, X. Zhang, M. Hu, Y. Yang, J. Yu, and X. Chen, “Repair or resample? Rethinking failure debugging in LLM multi-agent systems,” arXiv:2608.25920, 2026.